Implementación y Evaluación de K-means Paralelo con OpenMP

Malik Corvera Choi, Rubén Eduardo Bernal Puente

1. Introducción

El presente proyecto tiene como objetivo implementar y evaluar el desempeño del algoritmo de clustering K-means en C++, incorporando técnicas de paralelización mediante OpenMP y un entorno de ejecución inicializado con MPI. El propósito principal es analizar el impacto del paralelismo en términos de tiempo de ejecución, speedup y eficiencia, comparando directamente los resultados contra una versión serial del mismo algoritmo.

Para ello, se diseñó un flujo de trabajo completo que incluye la generación de datos sintéticos, la ejecución de experimentos bajo distintas configuraciones de hilos, y la visualización de resultados mediante herramientas en Python. Este enfoque permite no solo medir mejoras en desempeño, sino también validar que la calidad del clustering se mantiene consistente entre ambas versiones.

2. Descripción del algoritmo

El algoritmo K-means consiste en un proceso iterativo que inicia con la selección de k centroides, los cuales se distribuyen aleatoriamente sobre los datos. Posteriormente, cada punto es asignado al centroide más cercano utilizando la distancia euclidiana. Una vez realizada la asignación, los centroides son actualizados como el promedio de los puntos pertenecientes a cada cluster. Este proceso se repite hasta que no haya cambios en las asignaciones o se alcance un número máximo de iteraciones.

La distancia utilizada para la asignación de cada punto a su centroide más cercano es la distancia euclidiana. Para dos puntos x_1 y x_2 en un espacio de dimensión d, esta distancia se define como

$$d(x_1, x_2) = \sqrt{\sum_{i=1}^{d} (x_{1i} - x_{2i})^2}.$$

En el contexto de este proyecto, el algoritmo trabaja con archivos CSV que contienen puntos en dos o tres dimensiones. El programa detecta automáticamente la dimensión del dataset a partir del número de columnas presentes en el archivo de entrada.

3. Descripción del código y estrategia de paralelización

La implementación principal fue desarrollada en C++. El programa permite ejecutar el algoritmo en modo serial, en modo paralelo y en modo experimental. En los modos serial y paralelo, se generan archivos CSV con las asignaciones finales de cluster para cada punto. En el modo experimental, el programa ejecuta múltiples repeticiones por configuración y produce un archivo con métricas agregadas de desempeño.

La estrategia de paralelización se concentra en las dos partes más costosas del algoritmo: la asignación de puntos a clusters y la actualización de centroides. En la primera, cada punto puede procesarse de manera independiente, por lo que se emplean bucles paralelos con OpenMP para calcular en paralelo la distancia entre cada punto y los centroides. En la segunda, se utilizan acumuladores parciales por hilo para sumar coordenadas y contar elementos por cluster, reduciendo así los conflictos de escritura en memoria compartida.

Para mantener la consistencia del algoritmo, la implementación utiliza mecanismos de reducción y consolidación de resultados parciales al final de cada iteración. Esto permite conservar la equivalencia lógica entre la versión serial y la versión paralela. MPI se utiliza para inicializar el entorno paralelo, aunque en la versión actual el trabajo efectivo se concentra en el proceso maestro, mientras OpenMP gestiona el paralelismo en memoria compartida.

4. Definición del experimento

La evaluación experimental se realizó utilizando datasets sintéticos generados en Python a partir de distribuciones gaussianas. Se trabajó con archivos en dos y tres dimensiones, manteniendo fijo el número de clusters en $k = 8$. Con el fin de evaluar escalabilidad respecto al tamaño del problema, se consideraron datasets de 100,000, 200,000, 300,000, 400,000, 600,000 y 800,000 puntos.

Para cada dataset se ejecutó el algoritmo con configuraciones de hilos $\{1, 4, 8, 16\}$. En cada caso se midieron tiempos de ejecución serial y paralelo, y a partir de ellos se calcularon

métricas de speedup, eficiencia y costo paralelo. Asimismo, se registraron valores promedio de WCSS e iteraciones, con el fin de verificar que la calidad de la solución no se modificara por efecto de la paralelización.

La salida experimental se almacenó en un archivo CSV de métricas, que posteriormente fue procesado con Python para generar gráficas comparativas de tiempo, speedup y eficiencia. Este diseño experimental permite comparar no solo el efecto del número de hilos, sino también el comportamiento de la implementación conforme crece el tamaño de los datos.

5. Equipo de cómputo y software

Los experimentos se realizaron en una computadora personal con arquitectura Apple Silicon. La compilación del programa se realizó con herramientas basadas en clang, utilizando la biblioteca `libomp` para habilitar OpenMP y OpenMPI para la inicialización del entorno MPI. Para el procesamiento y la visualización de resultados se empleó un entorno de Python con bibliotecas como `numpy`, `pandas` y `matplotlib`.

Esta configuración es adecuada para evaluar paralelismo en memoria compartida y permite analizar el comportamiento del algoritmo frente a diferentes configuraciones de hilos sin cambiar de entorno de ejecución.

6. Resultados experimentales

Los resultados muestran una reducción significativa en el tiempo de ejecución al incrementar el número de hilos, particularmente al pasar de 1 a 4 hilos. En prácticamente todos los tamaños de entrada se observa una caída fuerte en el tiempo paralelo, lo cual indica que la paralelización logra atacar con éxito la parte dominante del costo computacional. Sin embargo, a partir de 8 hilos las mejoras adicionales son menores, lo que sugiere la existencia de un punto de saturación.

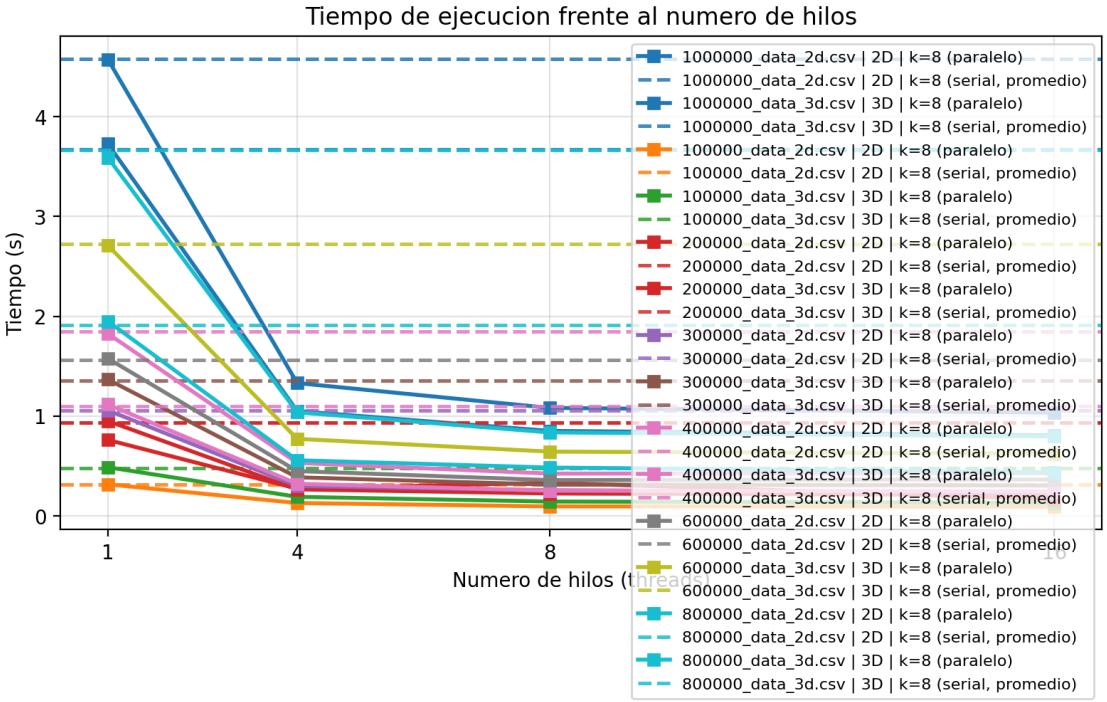

Figura 1: Tiempo de ejecución en función del número de hilos para distintos tamaños de datos y dimensiones. Las líneas punteadas representan el tiempo serial promedio de cada configuración.

Como se observa en la Figura 1, el tiempo de ejecución crece con el tamaño del dataset, como era de esperarse, pero al mismo tiempo la reducción absoluta de tiempo obtenida mediante paralelización también se vuelve más notoria en problemas grandes. Esto es importante porque muestra que el beneficio práctico del paralelismo es mayor cuando la carga computacional es suficientemente alta.

En términos de speedup, el comportamiento también es favorable. Para datasets pequeños, el speedup crece de manera clara de 1 a 4 y de 4 a 8 hilos, aunque luego tiende a estabilizarse. Para datasets más grandes, en cambio, el crecimiento es más sostenido, alcanzando valores superiores a 4 e incluso cercanos a 5 en algunos casos.

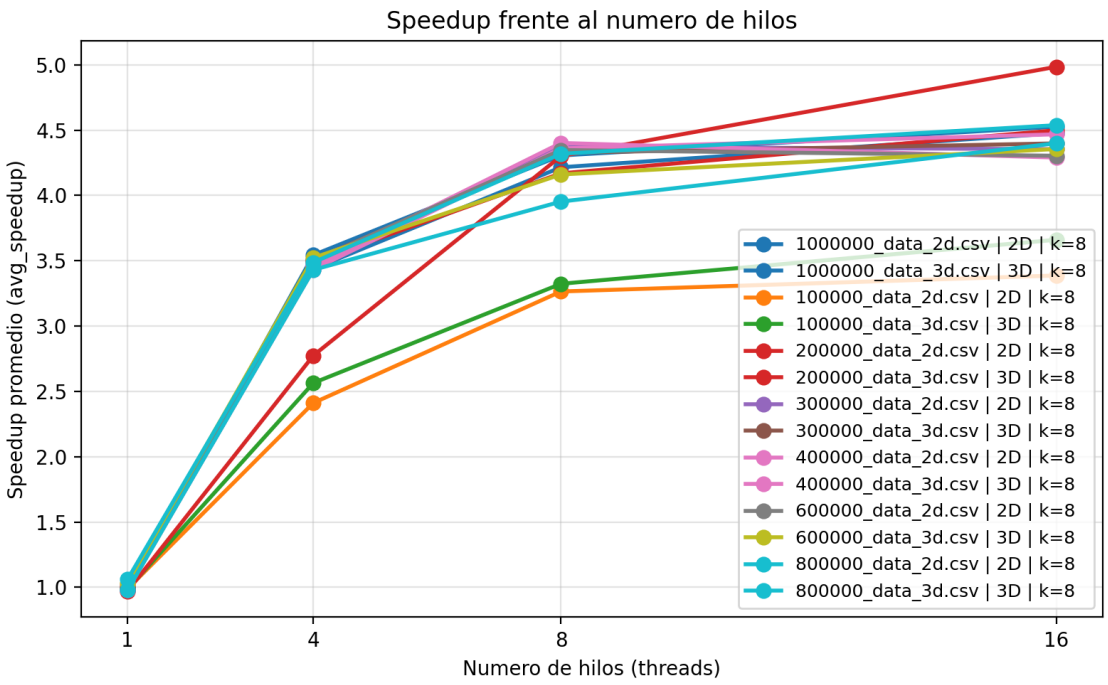

Figura 2: Speedup en función del número de hilos para distintos tamaños de datos y dimensiones.

La Figura 2 permite ver que la implementación no solo supera ampliamente el umbral mínimo de speedup requerido, sino que además presenta una escalabilidad razonable frente al crecimiento del tamaño de entrada. Los mejores resultados se observan en datasets medianos y grandes, donde el costo de sincronización se amortiza mejor en relación con el trabajo útil.

La eficiencia paralela, por su parte, presenta una tendencia decreciente conforme aumenta el número de hilos. Este comportamiento es esperado, ya que la eficiencia divide el speedup entre el número de hilos y, por lo tanto, penaliza configuraciones donde el crecimiento del speedup deja de ser proporcional al aumento de recursos.

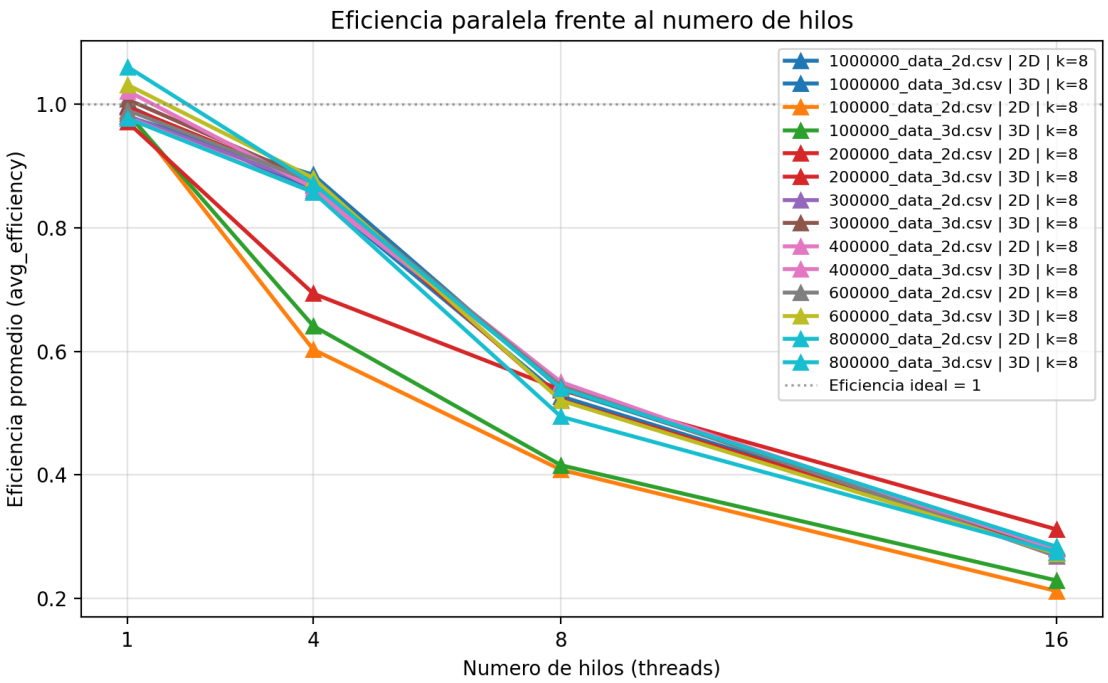

Figura 3: Eficiencia paralela en función del número de hilos para distintos tamaños de datos y dimensiones.

Como se aprecia en la Figura 3, la eficiencia es alta en configuraciones pequeñas e intermedias, especialmente en 4 hilos, y disminuye al llegar a 8 y 16 hilos. Aun así, los datasets grandes conservan niveles relativamente más altos de eficiencia, lo cual sugiere que la implementación aprovecha mejor el paralelismo cuando el volumen de trabajo es mayor.

Además del desempeño temporal, se verificó que la calidad del clustering no cambiara entre la versión serial y la versión paralela. Los experimentos mostraron coincidencia en los valores de WCSS y en el número de iteraciones promedio, lo que confirma que la paralelización no altera el resultado del algoritmo, sino únicamente su tiempo de ejecución.

7. Análisis de resultados

El comportamiento observado es consistente con la teoría clásica de escalabilidad. En particular, la mejora inicial fuerte al pasar de 1 a 4 hilos indica que una fracción importante del código fue efectivamente paralelizada. Sin embargo, la desaceleración de las ganancias a partir de 8 hilos sugiere la presencia de factores limitantes como sincronización, acceso a memoria compartida y secciones del algoritmo que permanecen esencialmente seriales.

Este patrón puede interpretarse a la luz de la Ley de Amdahl. Aunque el algoritmo se beneficia del paralelismo, el speedup máximo alcanzable está acotado por la parte del

programa que no puede distribuirse entre hilos. Por ello, aun cuando se incrementa el número de hilos, la eficiencia tiende a disminuir y las mejoras marginales se reducen.

También resulta relevante que los datasets más grandes muestran mejor comportamiento en speedup y eficiencia. Esto indica que el overhead del paralelismo se vuelve relativamente menos importante cuando el trabajo total es mayor. En otras palabras, la implementación es especialmente útil en escenarios donde el tamaño del problema justifica el costo adicional de coordinar múltiples hilos.

En conjunto, los resultados muestran que el rango más balanceado de operación se encuentra entre 4 y 8 hilos. En ese intervalo se consigue una reducción importante del tiempo de ejecución sin una pérdida tan marcada de eficiencia. Aunque 16 hilos puede seguir ofreciendo mejoras en algunos casos, estas ya no son tan proporcionales como en configuraciones intermedias.

8. Conclusiones

La implementación paralela del algoritmo K-means desarrollada en este proyecto logra mejoras sustanciales de desempeño respecto a la versión serial. El uso de OpenMP permite reducir de forma significativa el tiempo de ejecución, alcanzando speedups elevados y manteniendo la calidad del clustering sin modificaciones.

El análisis experimental también muestra que el beneficio del paralelismo depende tanto del número de hilos como del tamaño del dataset. En problemas más grandes, la implementación escala mejor y aprovecha de forma más eficiente los recursos computacionales disponibles. En contraste, cuando el número de hilos crece demasiado respecto al tamaño del problema, el overhead de sincronización y acceso a memoria limita las ganancias posibles.

En términos generales, puede concluirse que la estrategia de paralelización implementada es correcta, efectiva y adecuada para arquitecturas de memoria compartida. El proyecto cumple con el objetivo de comparar rigurosamente una versión serial contra una paralelizada, y demuestra experimentalmente que el paralelismo sí aporta mejoras medibles y relevantes en el contexto de K-means.